

ig - SYNTAX ERROR

St x i v e n

a 2 z

D S A S H E E T

- " 4 9 5 "

By - SYNTAX ERROR

Step 11: Binary Trees

The PDF created by Abhishek Rathor (Instagram:SYNTAX ERROR)

PROBLEM:1

Problem Statement:

Write a Java program to represent a binary tree with basic insertion logic.

Java Code:

```
class Node {
    int data;
    Node left, right;

    Node(int item) {
        data = item;
        left = right = null;
    }
}

public class BinaryTreeRepresentation {
    Node root;

    public static void main(String[] args) {
        BinaryTreeRepresentation tree = new BinaryTreeRepresentation();
        tree.root = new Node(1);
        tree.root.left = new Node(2);
        tree.root.right = new Node(3);
        tree.root.left.left = new Node(4);
        tree.root.left.right = new Node(5);

        System.out.println("Binary Tree created successfully!");
    }
}
```

Explanation:

1. A `Node` class is created to represent each node of the binary tree.
 2. The `BinaryTreeRepresentation` class has a `root` node.
 3. Nodes are manually assigned to build a binary tree.
-

PROBLEM:2

Problem Statement:

Perform Inorder Traversal of a Binary Tree (Left → Root → Right)

Java Code:

```
class Node {
    int data;
    Node left, right;

    Node(int item) {
        data = item;
        left = right = null;
    }
}

public class InorderTraversal {
    Node root;

    void inorder(Node node) {
        if (node == null)
            return;

        inorder(node.left);
        System.out.print(node.data + " ");
        inorder(node.right);
    }

    public static void main(String[] args) {
        InorderTraversal tree = new InorderTraversal();
        tree.root = new Node(1);
        tree.root.left = new Node(2);
        tree.root.right = new Node(3);
        tree.root.left.left = new Node(4);
        tree.root.left.right = new Node(5);

        System.out.print("Inorder traversal: ");
        tree.inorder(tree.root);
    }
}
```

Explanation:

1. Inorder traversal visits the left child, then the root, and then the right child.
 2. This uses recursion to traverse the binary tree in the specified order.
-

PROBLEM:3**Problem Statement:**

Write a Java program to perform **Preorder Traversal** of a Binary Tree (Root → Left → Right)

Java Code:

```
java
CopyEdit
class Node {
    int data;
    Node left, right;

    Node(int item) {
        data = item;
        left = right = null;
    }
}

public class PreorderTraversal {
    Node root;

    void preorder(Node node) {
        if (node == null)
            return;

        System.out.print(node.data + " ");
        preorder(node.left);
        preorder(node.right);
    }

    public static void main(String[] args) {
        PreorderTraversal tree = new PreorderTraversal();
        tree.root = new Node(1);
        tree.root.left = new Node(2);
        tree.root.right = new Node(3);
        tree.root.left.left = new Node(4);
        tree.root.left.right = new Node(5);

        System.out.print("Preorder traversal: ");
        tree.preorder(tree.root);
    }
}
```

Explanation:

1. In preorder traversal, we visit the root node first, then the left subtree, and finally the right subtree.
 2. This recursive approach makes it easy to handle.
-

PROBLEM:4

Problem Statement:

Write a Java program to perform **Postorder Traversal** of a Binary Tree (Left → Right → Root)

Java Code:

```
java
CopyEdit
class Node {
    int data;
    Node left, right;

    Node(int item) {
        data = item;
        left = right = null;
    }
}

public class PostorderTraversal {
    Node root;

    void postorder(Node node) {
        if (node == null)
            return;

        postorder(node.left);
        postorder(node.right);
        System.out.print(node.data + " ");
    }

    public static void main(String[] args) {
        PostorderTraversal tree = new PostorderTraversal();
        tree.root = new Node(1);
        tree.root.left = new Node(2);
        tree.root.right = new Node(3);
        tree.root.left.left = new Node(4);
        tree.root.left.right = new Node(5);

        System.out.print("Postorder traversal: ");
        tree.postorder(tree.root);
    }
}
```

Explanation:

1. Postorder traversal means visiting left, then right subtree, then the root.
 2. Useful in deleting the tree or evaluating postfix expressions.
-

PROBLEM:5

Problem Statement:

Write a Java program for **Level Order Traversal** (Breadth-First Traversal) of a Binary Tree.

Java Code:

```

java
CopyEdit
import java.util.*;

class Node {
    int data;
    Node left, right;

    Node(int item) {
        data = item;
        left = right = null;
    }
}

public class LevelOrderTraversal {
    Node root;

    void levelOrder(Node node) {
        if (node == null)
            return;

        Queue<Node> queue = new LinkedList<>();
        queue.add(node);

        while (!queue.isEmpty()) {
            Node temp = queue.poll();
            System.out.print(temp.data + " ");

            if (temp.left != null)
                queue.add(temp.left);

            if (temp.right != null)
                queue.add(temp.right);
        }

        public static void main(String[] args) {
            LevelOrderTraversal tree = new LevelOrderTraversal();
            tree.root = new Node(1);
            tree.root.left = new Node(2);
            tree.root.right = new Node(3);
            tree.root.left.left = new Node(4);
            tree.root.left.right = new Node(5);

            System.out.print("Level Order traversal: ");
            tree.levelOrder(tree.root);
        }
    }
}

```

Explanation:

1. Level Order Traversal uses a **queue** to visit nodes level by level.
2. It's also called **Breadth-First Search (BFS)** for trees.

PROBLEM:6

Problem Statement:

Write a Java program for **Level Order Traversal in Spiral Form** (also known as Zigzag Traversal).

Java Code:

```
java
CopyEdit
import java.util.*;

class Node {
    int data;
    Node left, right;
    Node(int item) {
        data = item;
        left = right = null;
    }
}

public class SpiralLevelOrder {
    Node root;

    void spiralOrder(Node node) {
        if (node == null)
            return;

        Stack<Node> s1 = new Stack<>();
        Stack<Node> s2 = new Stack<>();

        s1.push(node);

        while (!s1.isEmpty() || !s2.isEmpty()) {
            while (!s1.isEmpty()) {
                Node temp = s1.pop();
                System.out.print(temp.data + " ");
                if (temp.left != null)
                    s2.push(temp.left);
                if (temp.right != null)
                    s2.push(temp.right);
            }

            while (!s2.isEmpty()) {
                Node temp = s2.pop();
                System.out.print(temp.data + " ");
                if (temp.right != null)
                    s1.push(temp.right);
                if (temp.left != null)
                    s1.push(temp.left);
            }
        }
    }

    public static void main(String[] args) {
        SpiralLevelOrder tree = new SpiralLevelOrder();
        tree.root = new Node(1);
        tree.root.left = new Node(2);
        tree.root.right = new Node(3);
        tree.root.left.left = new Node(7);
    }
}
```

```

        tree.root.left.right = new Node(6);
        tree.root.right.left = new Node(5);
        tree.root.right.right = new Node(4);

        System.out.print("Spiral Order traversal: ");
        tree.spiralOrder(tree.root);
    }
}

```

Explanation:

1. We use **two stacks** to alternate between left-to-right and right-to-left traversals.
 2. Stack *s1* prints left to right; stack *s2* prints right to left.
-

PROBLEM:7

Problem Statement:

Write a Java program for **Iterative Preorder Traversal** of a Binary Tree.

Java Code:

```

java
CopyEdit
import java.util.*;

class Node {
    int data;
    Node left, right;
    Node(int item) {
        data = item;
        left = right = null;
    }
}

public class IterativePreorder {
    Node root;

    void iterativePreorder(Node node) {
        if (node == null)
            return;

        Stack<Node> stack = new Stack<>();
        stack.push(node);

        while (!stack.isEmpty()) {
            Node temp = stack.pop();
            System.out.print(temp.data + " ");

            if (temp.right != null)
                stack.push(temp.right);
            if (temp.left != null)
                stack.push(temp.left);
        }
    }
}

```



```

    }
}

public static void main(String[] args) {
    IterativePreorder tree = new IterativePreorder();
    tree.root = new Node(10);
    tree.root.left = new Node(20);
    tree.root.right = new Node(30);
    tree.root.left.left = new Node(40);
    tree.root.left.right = new Node(50);

    System.out.print("Iterative Preorder: ");
    tree.iterativePreorder(tree.root);
}
}

```

Explanation:

1. We use a **stack** to simulate the recursive call stack.
 2. Push right child before left child so that left is processed first.
-

PROBLEM:8

Problem Statement:

Write a Java program for **Iterative Inorder Traversal** of a Binary Tree.

Java Code:

```

java
CopyEdit
import java.util.*;

class Node {
    int data;
    Node left, right;
    Node(int item) {
        data = item;
        left = right = null;
    }
}

public class IterativeInorder {
    Node root;

    void iterativeInorder(Node node) {
        Stack<Node> stack = new Stack<>();
        Node current = node;

        while (current != null || !stack.isEmpty()) {
            while (current != null) {
                stack.push(current);
                current = current.left;
            }

```

```

        }

        current = stack.pop();
        System.out.print(current.data + " ");
        current = current.right;
    }
}

public static void main(String[] args) {
    IterativeInorder tree = new IterativeInorder();
    tree.root = new Node(1);
    tree.root.left = new Node(2);
    tree.root.right = new Node(3);
    tree.root.left.left = new Node(4);
    tree.root.left.right = new Node(5);

    System.out.print("Iterative Inorder: ");
    tree.iterativeInorder(tree.root);
}
}

```

Explanation:

1. Use a stack to traverse down the left side.
2. Pop and process nodes when left subtree is done, then go to right.

PROBLEM:9

Problem Statement:

Write a Java program for **Post-order Traversal using Two Stacks**.

Java Code:

```

java
CopyEdit
import java.util.*;

class Node {
    int data;
    Node left, right;
    Node(int val) {
        data = val;
        left = right = null;
    }
}

public class PostOrderTwoStacks {
    Node root;

    void postOrderTwoStacks(Node node) {
        if (node == null) return;
    }
}

```

```

Stack<Node> s1 = new Stack<>();
Stack<Node> s2 = new Stack<>();

s1.push(node);
while (!s1.isEmpty()) {
    Node temp = s1.pop();
    s2.push(temp);
    if (temp.left != null)
        s1.push(temp.left);
    if (temp.right != null)
        s1.push(temp.right);
}

while (!s2.isEmpty()) {
    System.out.print(s2.pop().data + " ");
}

}

public static void main(String[] args) {
    PostOrderTwoStacks tree = new PostOrderTwoStacks();
    tree.root = new Node(1);
    tree.root.left = new Node(2);
    tree.root.right = new Node(3);
    tree.root.left.left = new Node(4);
    tree.root.left.right = new Node(5);

    System.out.print("Postorder using 2 stacks: ");
    tree.postOrderTwoStacks(tree.root);
}
}

```

Explanation:

- First stack stores nodes in root-right-left order.
 - Second stack reverses this to left-right-root.
-

PROBLEM:10

Problem Statement:

Write a Java program for **Post-order Traversal using One Stack**.

Java Code:

```

java
CopyEdit
import java.util.*;

class Node {
    int data;
    Node left, right;
    Node(int val) {
        data = val;
    }
}

```

```

        left = right = null;
    }
}

public class PostOrderOneStack {
    Node root;

    void postOrderOneStack(Node node) {
        Stack<Node> stack = new Stack<>();
        Node current = node, lastVisited = null;

        while (current != null || !stack.isEmpty()) {
            if (current != null) {
                stack.push(current);
                current = current.left;
            } else {
                Node peekNode = stack.peek();
                if (peekNode.right != null && lastVisited != peekNode.right)
                {
                    current = peekNode.right;
                } else {
                    System.out.print(peekNode.data + " ");
                    lastVisited = stack.pop();
                }
            }
        }
    }

    public static void main(String[] args) {
        PostOrderOneStack tree = new PostOrderOneStack();
        tree.root = new Node(1);
        tree.root.left = new Node(2);
        tree.root.right = new Node(3);
        tree.root.left.left = new Node(4);
        tree.root.left.right = new Node(5);

        System.out.print("Postorder using 1 stack: ");
        tree.postOrderOneStack(tree.root);
    }
}

```

Explanation:

- Stack is used to simulate recursion.
 - Track last visited to avoid reprocessing right subtree.
-

PROBLEM:11

Problem Statement:

Write a Java program to get **Preorder, Inorder, and Postorder Traversals in one traversal**.

Java Code:

```

java
CopyEdit
import java.util.*;

class Node {
    int data;
    Node left, right;
    Node(int val) {
        data = val;
        left = right = null;
    }
}

class Pair {
    Node node;
    int state;
    Pair(Node n, int s) {
        node = n;
        state = s;
    }
}

public class AllTraversalsOnePass {
    Node root;

    void allTraversals(Node node) {
        Stack<Pair> stack = new Stack<>();
        List<Integer> pre = new ArrayList<>();
        List<Integer> in = new ArrayList<>();
        List<Integer> post = new ArrayList<>();

        stack.push(new Pair(node, 1));

        while (!stack.isEmpty()) {
            Pair p = stack.pop();

            if (p.state == 1) {
                pre.add(p.node.data);
                p.state++;
                stack.push(p);
                if (p.node.left != null)
                    stack.push(new Pair(p.node.left, 1));
            } else if (p.state == 2) {
                in.add(p.node.data);
                p.state++;
                stack.push(p);
                if (p.node.right != null)
                    stack.push(new Pair(p.node.right, 1));
            } else {
                post.add(p.node.data);
            }
        }

        System.out.println("Preorder: " + pre);
        System.out.println("Inorder: " + in);
        System.out.println("Postorder: " + post);
    }

    public static void main(String[] args) {
        AllTraversalsOnePass tree = new AllTraversalsOnePass();
        tree.root = new Node(1);
    }
}

```

```
        tree.root.left = new Node(2);
        tree.root.right = new Node(3);
        tree.root.left.left = new Node(4);
        tree.root.left.right = new Node(5);

        tree.allTraversals(tree.root);
    }
}
```

Explanation:

- We use a stack and a state variable for each node.
 - 1: Preorder
 - 2: Inorder
 - 3: Postorder

PROBLEM:12

Problem Statement:

Write a Java program to **find the Height of a Binary Tree**.

Java Code:

```
java
CopyEdit
class Node {
    int data;
    Node left, right;
    Node(int val) {
        data = val;
        left = right = null;
    }
}

public class HeightOfTree {
    Node root;

    int height(Node node) {
        if (node == null) return 0;
        return 1 + Math.max(height(node.left), height(node.right));
    }

    public static void main(String[] args) {
        HeightOfTree tree = new HeightOfTree();
        tree.root = new Node(1);
        tree.root.left = new Node(2);
        tree.root.right = new Node(3);
        tree.root.left.left = new Node(4);

        System.out.println("Height of tree: " + tree.height(tree.root));
    }
}
```

Explanation:

- Height = max depth from root to leaf.
 - Base case returns 0 for null.
-

PROBLEM:13

Problem Statement:

Write a Java program to calculate the **Diameter of a Binary Tree** (longest path between two nodes).

Java Code:

```
java
CopyEdit
class Node {
    int data;
    Node left, right;
    Node(int val) {
        data = val;
        left = right = null;
    }
}

public class DiameterOfTree {
    Node root;
    int diameter = 0;

    int dfs(Node node) {
        if (node == null) return 0;
        int left = dfs(node.left);
        int right = dfs(node.right);
        diameter = Math.max(diameter, left + right);
        return 1 + Math.max(left, right);
    }

    int getDiameter(Node root) {
        dfs(root);
        return diameter;
    }

    public static void main(String[] args) {
        DiameterOfTree tree = new DiameterOfTree();
        tree.root = new Node(1);
        tree.root.left = new Node(2);
        tree.root.right = new Node(3);
        tree.root.left.left = new Node(4);
        tree.root.left.right = new Node(5);

        System.out.println("Diameter: " + tree.getDiameter(tree.root));
    }
}
```

Explanation:

- Diameter = max path length through a node (left + right subtree heights).
 - DFS used to calculate height + update global max.
-

PROBLEM:14

Problem Statement:

Check if a binary tree is a **mirror (symmetric) of itself**.

Java Code:

```
java
CopyEdit
class Node {
    int data;
    Node left, right;
    Node(int val) {
        data = val;
        left = right = null;
    }
}

public class SymmetricTree {
    Node root;

    boolean isMirror(Node t1, Node t2) {
        if (t1 == null && t2 == null) return true;
        if (t1 == null || t2 == null) return false;
        return (t1.data == t2.data) &&
            isMirror(t1.left, t2.right) &&
            isMirror(t1.right, t2.left);
    }

    boolean isSymmetric(Node root) {
        return isMirror(root, root);
    }

    public static void main(String[] args) {
        SymmetricTree tree = new SymmetricTree();
        tree.root = new Node(1);
        tree.root.left = new Node(2);
        tree.root.right = new Node(2);
        tree.root.left.left = new Node(3);
        tree.root.right.right = new Node(3);

        System.out.println("Is Symmetric? " + tree.isSymmetric(tree.root));
    }
}
```

Explanation:

- Use two pointers moving inward to compare left & right subtrees.

PROBLEM:15

Problem Statement:

Find the **Maximum Path Sum** in a Binary Tree (can start & end at any node).

Java Code:

```
java
CopyEdit
class Node {
    int data;
    Node left, right;
    Node(int val) {
        data = val;
        left = right = null;
    }
}

public class MaxPathSum {
    Node root;
    int maxSum = Integer.MIN_VALUE;

    int maxGain(Node node) {
        if (node == null) return 0;
        int left = Math.max(0, maxGain(node.left));
        int right = Math.max(0, maxGain(node.right));
        maxSum = Math.max(maxSum, left + right + node.data);
        return node.data + Math.max(left, right);
    }

    int findMaxPath(Node root) {
        maxGain(root);
        return maxSum;
    }

    public static void main(String[] args) {
        MaxPathSum tree = new MaxPathSum();
        tree.root = new Node(-10);
        tree.root.left = new Node(9);
        tree.root.right = new Node(20);
        tree.root.right.left = new Node(15);
        tree.root.right.right = new Node(7);

        System.out.println("Maximum Path Sum: " +
            tree.findMaxPath(tree.root));
    }
}
```

Explanation:

- At every node, calculate `left + right + node value`.
 - Only return 1 child (left/right) to parent as path can't split.
-

PROBLEM:16

Problem Statement:

Find the **Lowest Common Ancestor (LCA)** of two nodes in a Binary Tree.

Java Code:

```
java
CopyEdit
class Node {
    int data;
    Node left, right;
    Node(int val) {
        data = val;
        left = right = null;
    }
}

public class LowestCommonAncestor {
    Node root;

    Node lca(Node root, int n1, int n2) {
        if (root == null) return null;
        if (root.data == n1 || root.data == n2) return root;

        Node left = lca(root.left, n1, n2);
        Node right = lca(root.right, n1, n2);

        if (left != null && right != null) return root;
        return (left != null) ? left : right;
    }

    public static void main(String[] args) {
        LowestCommonAncestor tree = new LowestCommonAncestor();
        tree.root = new Node(3);
        tree.root.left = new Node(5);
        tree.root.right = new Node(1);
        tree.root.left.left = new Node(6);
        tree.root.left.right = new Node(2);
        tree.root.right.left = new Node(0);
        tree.root.right.right = new Node(8);

        Node ancestor = tree.lca(tree.root, 5, 1);
        System.out.println("LCA: " + (ancestor != null ? ancestor.data :
"Not found"));
    }
}
```

Explanation:

- If one node is in left subtree and other in right → current is LCA.
- Recurse down to find presence of both.

Great! Let's continue with solutions for the next set of **Binary Tree Problems** in the same format. 🚀

Starting from where we left off:

PROBLEM:17

Problem Statement:

Check if the **Binary Tree is Height-Balanced** or not.

(A tree is balanced if for every node, height difference between left and right subtree is not more than 1.)

Java Code:

```
class Node {
    int data;
    Node left, right;
    Node(int val) { data = val; left = right = null; }
}

class BalancedBinaryTree {
    boolean isBalanced = true;

    int check(Node node) {
        if (node == null) return 0;
        int left = check(node.left);
        int right = check(node.right);
        if (Math.abs(left - right) > 1) isBalanced = false;
        return 1 + Math.max(left, right);
    }

    boolean isHeightBalanced(Node root) {
        check(root);
        return isBalanced;
    }

    public static void main(String[] args) {
        BalancedBinaryTree tree = new BalancedBinaryTree();
        Node root = new Node(1);
        root.left = new Node(2);
        root.left.left = new Node(3);
        root.right = new Node(4);

        System.out.println(tree.isHeightBalanced(root)); // false
    }
}
```

PROBLEM:18

Problem Statement:

Check if two **binary trees** are identical or not.

Java Code:

```
class Node {
    int data;
    Node left, right;
    Node(int val) { data = val; left = right = null; }
}

class IdenticalTrees {
    boolean isIdentical(Node a, Node b) {
        if (a == null && b == null) return true;
        if (a == null || b == null) return false;
        return (a.data == b.data) &&
            isIdentical(a.left, b.left) &&
            isIdentical(a.right, b.right);
    }

    public static void main(String[] args) {
        Node root1 = new Node(1);
        root1.left = new Node(2);
        root1.right = new Node(3);

        Node root2 = new Node(1);
        root2.left = new Node(2);
        root2.right = new Node(3);

        IdenticalTrees obj = new IdenticalTrees();
        System.out.println(obj.isIdentical(root1, root2)); // true
    }
}
```

PROBLEM:19**Problem Statement:**

Perform **Zig Zag (spiral) Traversal** of Binary Tree.

Java Code:

```
import java.util.*;

class Node {
    int data;
    Node left, right;
    Node(int val) { data = val; left = right = null; }
}

class ZigZagTraversal {
    List<List<Integer>> zigzagLevelOrder(Node root) {
        List<List<Integer>> res = new ArrayList<>();
    }
}
```

```

        if (root == null) return res;
        Queue<Node> q = new LinkedList<>();
        q.add(root);
        boolean leftToRight = true;

        while (!q.isEmpty()) {
            int size = q.size();
            List<Integer> level = new ArrayList<>(Collections.nCopies(size,
0));

            for (int i = 0; i < size; i++) {
                Node node = q.poll();
                int index = leftToRight ? i : size - 1 - i;
                level.set(index, node.data);

                if (node.left != null) q.add(node.left);
                if (node.right != null) q.add(node.right);
            }
            leftToRight = !leftToRight;
            res.add(level);
        }
        return res;
    }

    public static void main(String[] args) {
        ZigZagTraversal t = new ZigZagTraversal();
        Node root = new Node(1);
        root.left = new Node(2); root.right = new Node(3);
        root.left.left = new Node(4); root.left.right = new Node(5);
        root.right.left = new Node(6); root.right.right = new Node(7);

        System.out.println(t.zigzagLevelOrder(root));
    }
}

```

PROBLEM:20

Problem Statement:

Boundary Traversal of a Binary Tree.

(Print left boundary, leaf nodes, then right boundary in reverse)

Java Code:

```

java
CopyEdit
import java.util.*;

class Node {
    int data;
    Node left, right;
    Node(int val) { data = val; left = right = null; }
}

class BoundaryTraversal {
    List<Integer> res = new ArrayList<>();
}

```

```

void addLeft(Node root) {
    Node curr = root.left;
    while (curr != null) {
        if (!(curr.left == null && curr.right == null))
            res.add(curr.data);
        if (curr.left != null) curr = curr.left;
        else curr = curr.right;
    }
}

void addLeaves(Node root) {
    if (root == null) return;
    if (root.left == null && root.right == null) {
        res.add(root.data);
        return;
    }
    addLeaves(root.left);
    addLeaves(root.right);
}

void addRight(Node root) {
    Node curr = root.right;
    List<Integer> temp = new ArrayList<>();
    while (curr != null) {
        if (!(curr.left == null && curr.right == null))
            temp.add(curr.data);
        if (curr.right != null) curr = curr.right;
        else curr = curr.left;
    }
    Collections.reverse(temp);
    res.addAll(temp);
}

List<Integer> boundaryTraversal(Node root) {
    if (root == null) return res;
    if (!(root.left == null && root.right == null)) res.add(root.data);
    addLeft(root);
    addLeaves(root);
    addRight(root);
    return res;
}

public static void main(String[] args) {
    Node root = new Node(1);
    root.left = new Node(2); root.right = new Node(3);
    root.left.left = new Node(4); root.left.right = new Node(5);
    root.right.left = new Node(6); root.right.right = new Node(7);

    BoundaryTraversal obj = new BoundaryTraversal();
    System.out.println(obj.boundaryTraversal(root)); // [1, 2, 4, 5, 6,
7, 3]
}

```

PROBLEM:21

Problem Statement:

Vertical Order Traversal of Binary Tree.

Java Code:

```
java
CopyEdit
import java.util.*;

class Node {
    int data;
    Node left, right;
    Node(int val) { data = val; left = right = null; }
}

class VerticalOrder {
    class Pair {
        Node node; int hd;
        Pair(Node n, int h) { node = n; hd = h; }
    }

    List<List<Integer>> verticalOrder(Node root) {
        TreeMap<Integer, List<Integer>> map = new TreeMap<>();
        Queue<Pair> q = new LinkedList<>();
        q.add(new Pair(root, 0));

        while (!q.isEmpty()) {
            Pair p = q.poll();
            map.putIfAbsent(p.hd, new ArrayList<>());
            map.get(p.hd).add(p.node.data);

            if (p.node.left != null) q.add(new Pair(p.node.left, p.hd - 1));
            if (p.node.right != null) q.add(new Pair(p.node.right, p.hd +
1));
        }

        return new ArrayList<>(map.values());
    }

    public static void main(String[] args) {
        Node root = new Node(1);
        root.left = new Node(2); root.right = new Node(3);
        root.left.left = new Node(4); root.left.right = new Node(5);
        root.right.left = new Node(6); root.right.right = new Node(7);

        VerticalOrder obj = new VerticalOrder();
        System.out.println(obj.verticalOrder(root));
    }
}
```

PROBLEM:22

Problem Statement:

Top View of Binary Tree (First node seen from top at each horizontal distance)

Java Code:

```

java
CopyEdit
import java.util.*;

class Node {
    int data;
    Node left, right;
    Node(int val) { data = val; left = right = null; }
}

class TopViewBinaryTree {
    class Pair {
        Node node; int hd;
        Pair(Node n, int h) { node = n; hd = h; }
    }

    List<Integer> topView(Node root) {
        TreeMap<Integer, Integer> map = new TreeMap<>();
        Queue<Pair> q = new LinkedList<>();
        q.add(new Pair(root, 0));

        while (!q.isEmpty()) {
            Pair p = q.poll();
            if (!map.containsKey(p.hd))
                map.put(p.hd, p.node.data);

            if (p.node.left != null) q.add(new Pair(p.node.left, p.hd - 1));
            if (p.node.right != null) q.add(new Pair(p.node.right, p.hd +
1));
        }

        return new ArrayList<>(map.values());
    }

    public static void main(String[] args) {
        Node root = new Node(1);
        root.left = new Node(2); root.right = new Node(3);
        root.left.left = new Node(4); root.left.right = new Node(5);
        root.right.left = new Node(6); root.right.right = new Node(7);

        TopViewBinaryTree obj = new TopViewBinaryTree();
        System.out.println(obj.topView(root)); // [4, 2, 1, 3, 7]
    }
}

```

Great! Let's continue with the **remaining Binary Tree problems** in the same clean format:

PROBLEM:23

Problem Statement:

Bottom View of Binary Tree

(Print the last node seen from bottom at each horizontal distance)

Java Code:

```
import java.util.*;

class Node {
    int data;
    Node left, right;
    Node(int val) { data = val; left = right = null; }
}

class BottomViewBinaryTree {
    class Pair {
        Node node; int hd;
        Pair(Node n, int h) { node = n; hd = h; }
    }

    List<Integer> bottomView(Node root) {
        TreeMap<Integer, Integer> map = new TreeMap<>();
        Queue<Pair> q = new LinkedList<>();
        q.add(new Pair(root, 0));

        while (!q.isEmpty()) {
            Pair p = q.poll();
            map.put(p.hd, p.node.data); // overwrite

            if (p.node.left != null) q.add(new Pair(p.node.left, p.hd - 1));
            if (p.node.right != null) q.add(new Pair(p.node.right, p.hd +
1));
        }

        return new ArrayList<>(map.values());
    }

    public static void main(String[] args) {
        Node root = new Node(1);
        root.left = new Node(2); root.right = new Node(3);
        root.left.left = new Node(4); root.left.right = new Node(5);
        root.right.left = new Node(6); root.right.right = new Node(7);

        BottomViewBinaryTree obj = new BottomViewBinaryTree();
        System.out.println(obj.bottomView(root)); // [4, 2, 6, 3, 7]
    }
}
```

PROBLEM:24

Problem Statement:

Left View / Right View of Binary Tree

Java Code (Left View):

```
import java.util.*;

class Node {
    int data;
```

```

    Node left, right;
    Node(int val) { data = val; left = right = null; }
}

class LeftView {
    List<Integer> leftView(Node root) {
        List<Integer> res = new ArrayList<>();
        if (root == null) return res;

        Queue<Node> q = new LinkedList<>();
        q.add(root);

        while (!q.isEmpty()) {
            int size = q.size();
            for (int i = 0; i < size; i++) {
                Node temp = q.poll();
                if (i == 0) res.add(temp.data);

                if (temp.left != null) q.add(temp.left);
                if (temp.right != null) q.add(temp.right);
            }
        }
        return res;
    }
}

```

Java Code (Right View):

```

class RightView {
    List<Integer> rightView(Node root) {
        List<Integer> res = new ArrayList<>();
        if (root == null) return res;

        Queue<Node> q = new LinkedList<>();
        q.add(root);

        while (!q.isEmpty()) {
            int size = q.size();
            for (int i = 0; i < size; i++) {
                Node temp = q.poll();
                if (i == size - 1) res.add(temp.data);

                if (temp.left != null) q.add(temp.left);
                if (temp.right != null) q.add(temp.right);
            }
        }
        return res;
    }
}

```

PROBLEM:25

Problem Statement:

Check if a **Binary Tree is Symmetric** around its center.

Java Code:

```
class Solution {
    boolean isSymmetric(Node root) {
        return root == null || isMirror(root.left, root.right);
    }

    boolean isMirror(Node t1, Node t2) {
        if (t1 == null || t2 == null) return t1 == t2;
        if (t1.data != t2.data) return false;
        return isMirror(t1.left, t2.right) && isMirror(t1.right, t2.left);
    }
}
```

PROBLEM:26

Problem Statement:

Root to Node Path in Binary Tree

(Find path from root to a given node)

Java Code:

```
java
CopyEdit
class Solution {
    boolean getPath(Node root, int x, List<Integer> path) {
        if (root == null) return false;
        path.add(root.data);
        if (root.data == x) return true;

        if (getPath(root.left, x, path) || getPath(root.right, x, path))
            return true;

        path.remove(path.size() - 1);
        return false;
    }

    List<Integer> rootToNodePath(Node root, int x) {
        List<Integer> path = new ArrayList<>();
        getPath(root, x, path);
        return path;
    }
}
```

PROBLEM:27

Problem Statement:

LCA in Binary Tree (Lowest Common Ancestor)

Java Code:

```
java
CopyEdit
class Solution {
    Node lca(Node root, int n1, int n2) {
        if (root == null || root.data == n1 || root.data == n2)
            return root;

        Node left = lca(root.left, n1, n2);
        Node right = lca(root.right, n1, n2);

        if (left != null && right != null) return root;
        return left != null ? left : right;
    }
}
```

PROBLEM:28

Problem Statement:

Maximum Width of a Binary Tree

Java Code:

```
java
CopyEdit
class Solution {
    class Pair {
        Node node; int index;
        Pair(Node n, int i) { node = n; index = i; }
    }

    int widthOfBinaryTree(Node root) {
        if (root == null) return 0;
        int maxWidth = 0;
        Queue<Pair> q = new LinkedList<>();
        q.add(new Pair(root, 0));

        while (!q.isEmpty()) {
            int size = q.size();
            int minIndex = q.peek().index;
            int first = 0, last = 0;

            for (int i = 0; i < size; i++) {
                int currIndex = q.peek().index - minIndex;
                Node node = q.peek().node;
                q.poll();

                if (i == 0) first = currIndex;
                if (i == size - 1) last = currIndex;

                if (node.left != null) q.add(new Pair(node.left, 2 *
currIndex + 1));
                if (node.right != null) q.add(new Pair(node.right, 2 *
currIndex + 2));
            }
        }
        return last - first + 1;
    }
}
```

```

        }

        maxWidth = Math.max(maxWidth, last - first + 1);
    }

    return maxWidth;
}
}

```

Problem Statement:

Find the k th largest element in a stream of integers.

Java Code:

```

java
CopyEdit
import java.util.*;

class KthLargest {
    private PriorityQueue<Integer> minHeap;
    private int k;

    public KthLargest(int k, int[] nums) {
        this.k = k;
        minHeap = new PriorityQueue<>();
        for (int num : nums) {
            add(num);
        }
    }

    public int add(int val) {
        minHeap.add(val);
        if (minHeap.size() > k)
            minHeap.poll();
        return minHeap.peek();
    }
}

```

Explanation:

1. Maintain a min heap of size k .
2. Always keep k largest elements.
3. Top of heap is the k th largest element.

PROBLEM: 29

Problem Statement:

Write a Java program to convert an Infix expression to Postfix using Stack.

Java Code:

```
java
CopyEdit
import java.util.*;

public class InfixToPostfix {
    public static int precedence(char ch) {
        switch (ch) {
            case '+': case '-': return 1;
            case '*': case '/': return 2;
            case '^': return 3;
        }
        return -1;
    }

    public static String convert(String exp) {
        StringBuilder result = new StringBuilder();
        Stack<Character> stack = new Stack<>();

        for (int i = 0; i < exp.length(); i++) {
            char c = exp.charAt(i);

            if (Character.isLetterOrDigit(c)) {
                result.append(c);
            } else if (c == '(') {
                stack.push(c);
            } else if (c == ')') {
                while (!stack.isEmpty() && stack.peek() != '(')
                    result.append(stack.pop());
                stack.pop();
            } else {
                while (!stack.isEmpty() && precedence(c) <=
precedence(stack.peek()))
                    result.append(stack.pop());
                stack.push(c);
            }
        }

        while (!stack.isEmpty())
            result.append(stack.pop());

        return result.toString();
    }

    public static void main(String[] args) {
        String infix = "a+b*(c^d-e)^(f+g*h)-i";
        System.out.println("Postfix: " + convert(infix));
    }
}
```

Explanation:

1. Traverse the infix expression.
2. Add operands to the result.
3. Use stack to maintain operator precedence and handle parentheses.
4. Pop any remaining operators to result.

Sure! Let's continue with the next set of Binary Tree problems:

PROBLEM:29

Problem Statement:

Check for Children Sum Property

(A node's value should be equal to the sum of its children's values)

Java Code:

```
class Solution {
    public boolean isSumProperty(Node root) {
        if (root == null || (root.left == null && root.right == null))
            return true;

        int sum = 0;
        if (root.left != null) sum += root.left.data;
        if (root.right != null) sum += root.right.data;

        return (root.data == sum &&
                isSumProperty(root.left) &&
                isSumProperty(root.right));
    }
}
```

PROBLEM:30

Problem Statement:

Print all the Nodes at a Distance of K in a Binary Tree

Java Code:

```
class Solution {
    void markParents(Node root, Map<Node, Node> parentTrack) {
        Queue<Node> q = new LinkedList<>();
        q.offer(root);
        while (!q.isEmpty()) {
            Node current = q.poll();
            if (current.left != null) {
                parentTrack.put(current.left, current);
                q.offer(current.left);
            }
            if (current.right != null) {
                parentTrack.put(current.right, current);
                q.offer(current.right);
            }
        }
    }
}
```

```

    }
}

List<Integer> distanceK(Node root, Node target, int K) {
    Map<Node, Node> parentTrack = new HashMap<>();
    markParents(root, parentTrack);

    Queue<Node> q = new LinkedList<>();
    Map<Node, Boolean> visited = new HashMap<>();
    q.offer(target);
    visited.put(target, true);
    int curr_level = 0;

    while (!q.isEmpty()) {
        int size = q.size();
        if (curr_level == K) break;
        curr_level++;

        for (int i = 0; i < size; i++) {
            Node curr = q.poll();
            if (curr.left != null && !visited.containsKey(curr.left)) {
                q.offer(curr.left);
                visited.put(curr.left, true);
            }
            if (curr.right != null && !visited.containsKey(curr.right))
            {
                q.offer(curr.right);
                visited.put(curr.right, true);
            }
            Node parent = parentTrack.get(curr);
            if (parent != null && !visited.containsKey(parent)) {
                q.offer(parent);
                visited.put(parent, true);
            }
        }
    }

    List<Integer> result = new ArrayList<>();
    while (!q.isEmpty()) result.add(q.poll().data);
    return result;
}

```

PROBLEM:31

Problem Statement:

Minimum Time Taken to BURN the Binary Tree from a Node

Java Code:

```

class Solution {
    class Pair {
        Node node;
        int time;
    }
}

```



```

    Pair(Node node, int time) {
        this.node = node;
        this.time = time;
    }
}

int minTime(Node root, int target) {
    Map<Node, Node> parent = new HashMap<>();
    Node targetNode = markParentsAndFindTarget(root, parent, target);
    Map<Node, Boolean> visited = new HashMap<>();
    Queue<Pair> q = new LinkedList<>();
    q.add(new Pair(targetNode, 0));
    visited.put(targetNode, true);

    int maxTime = 0;
    while (!q.isEmpty()) {
        Pair p = q.poll();
        Node curr = p.node;
        int time = p.time;
        maxTime = Math.max(maxTime, time);

        if (curr.left != null && !visited.containsKey(curr.left)) {
            visited.put(curr.left, true);
            q.add(new Pair(curr.left, time + 1));
        }
        if (curr.right != null && !visited.containsKey(curr.right)) {
            visited.put(curr.right, true);
            q.add(new Pair(curr.right, time + 1));
        }
        if (parent.get(curr) != null
&& !visited.containsKey(parent.get(curr))) {
            visited.put(parent.get(curr), true);
            q.add(new Pair(parent.get(curr), time + 1));
        }
    }
    return maxTime;
}

Node markParentsAndFindTarget(Node root, Map<Node, Node> parent, int
target) {
    Queue<Node> q = new LinkedList<>();
    q.offer(root);
    Node targetNode = null;
    while (!q.isEmpty()) {
        Node curr = q.poll();
        if (curr.data == target) targetNode = curr;
        if (curr.left != null) {
            parent.put(curr.left, curr);
            q.offer(curr.left);
        }
        if (curr.right != null) {
            parent.put(curr.right, curr);
            q.offer(curr.right);
        }
    }
    return targetNode;
}
}

```

PROBLEM:32

Problem Statement:

Count Total Nodes in a COMPLETE Binary Tree

Java Code:

```
class Solution {
    public int countNodes(TreeNode root) {
        if (root == null) return 0;

        int leftHeight = getLeftHeight(root);
        int rightHeight = getRightHeight(root);

        if (leftHeight == rightHeight)
            return (1 << leftHeight) - 1;

        return 1 + countNodes(root.left) + countNodes(root.right);
    }

    private int getLeftHeight(TreeNode node) {
        int height = 0;
        while (node != null) {
            height++;
            node = node.left;
        }
        return height;
    }

    private int getRightHeight(TreeNode node) {
        int height = 0;
        while (node != null) {
            height++;
            node = node.right;
        }
        return height;
    }
}
```

PROBLEM:33

Problem Statement:

Requirements Needed to Construct a Unique Binary Tree

(Just theoretical understanding)

Theory:

To construct a **unique binary tree**, you need both:

- **Inorder traversal**
- **One of Preorder or Postorder traversal**

Why?

Because inorder provides **relative positioning** (left/right) and preorder/postorder helps in determining **root node order**.

PROBLEM:34

Problem Statement:

Construct Binary Tree from Inorder and Preorder Traversal

Java Code:

```
java
CopyEdit
class Solution {
    int preorderIndex = 0;
    Map<Integer, Integer> inorderMap = new HashMap<>();

    public TreeNode buildTree(int[] preorder, int[] inorder) {
        for (int i = 0; i < inorder.length; i++)
            inorderMap.put(inorder[i], i);

        return build(preorder, 0, inorder.length - 1);
    }

    private TreeNode build(int[] preorder, int left, int right) {
        if (left > right) return null;

        int rootVal = preorder[preorderIndex++];
        TreeNode root = new TreeNode(rootVal);

        int index = inorderMap.get(rootVal);
        root.left = build(preorder, left, index - 1);
        root.right = build(preorder, index + 1, right);

        return root;
    }
}
```

PROBLEM:35

Problem Statement:

Construct Binary Tree from Inorder and Postorder Traversal

Java Code:

```
java
CopyEdit
```

```

class Solution {
    int postIndex;
    Map<Integer, Integer> inorderMap = new HashMap<>();

    public TreeNode buildTree(int[] inorder, int[] postorder) {
        postIndex = postorder.length - 1;
        for (int i = 0; i < inorder.length; i++)
            inorderMap.put(inorder[i], i);

        return helper(postorder, 0, inorder.length - 1);
    }

    private TreeNode helper(int[] postorder, int left, int right) {
        if (left > right) return null;

        int rootVal = postorder[postIndex--];
        TreeNode root = new TreeNode(rootVal);

        int index = inorderMap.get(rootVal);
        root.right = helper(postorder, index + 1, right);
        root.left = helper(postorder, left, index - 1);

        return root;
    }
}

```

PROBLEM:36

Problem Statement:

Serialize and Deserialize Binary Tree

Java Code:

```

java
CopyEdit
class Codec {

    public String serialize(TreeNode root) {
        if (root == null) return "null,";
        return root.val + "," + serialize(root.left) +
serialize(root.right);
    }

    int index;
    public TreeNode deserialize(String data) {
        String[] values = data.split(",");
        index = 0;
        return helper(values);
    }

    private TreeNode helper(String[] values) {
        if (values[index].equals("null")) {
            index++;
            return null;
        }
    }
}

```

```
        TreeNode root = new TreeNode(Integer.parseInt(values[index++]));
        root.left = helper(values);
        root.right = helper(values);
        return root;
    }
}
```

PROBLEM:37

Problem Statement:

Morris Preorder Traversal of a Binary Tree

Java Code:

```
java
CopyEdit
class Solution {
    public List<Integer> morrisPreorder(TreeNode root) {
        List<Integer> result = new ArrayList<>();
        TreeNode curr = root;

        while (curr != null) {
            if (curr.left == null) {
                result.add(curr.val);
                curr = curr.right;
            } else {
                TreeNode prev = curr.left;
                while (prev.right != null && prev.right != curr)
                    prev = prev.right;

                if (prev.right == null) {
                    result.add(curr.val);
                    prev.right = curr;
                    curr = curr.left;
                } else {
                    prev.right = null;
                    curr = curr.right;
                }
            }
        }
        return result;
    }
}
```

PROBLEM:38

Problem Statement:

Morris Inorder Traversal of a Binary Tree

Java Code:

```
java
CopyEdit
class Solution {
    public List<Integer> morrisInorder(TreeNode root) {
        List<Integer> res = new ArrayList<>();
        TreeNode curr = root;

        while (curr != null) {
            if (curr.left == null) {
                res.add(curr.val);
                curr = curr.right;
            } else {
                TreeNode prev = curr.left;
                while (prev.right != null && prev.right != curr)
                    prev = prev.right;

                if (prev.right == null) {
                    prev.right = curr;
                    curr = curr.left;
                } else {
                    prev.right = null;
                    res.add(curr.val);
                    curr = curr.right;
                }
            }
        }
        return res;
    }
}
```

PROBLEM:39

Problem Statement:

Flatten Binary Tree to Linked List

(Transform binary tree into a right-skewed tree in-place)

Java Code:

```
java
CopyEdit
class Solution {
    public void flatten(TreeNode root) {
        TreeNode curr = root;
        while (curr != null) {
            if (curr.left != null) {
                TreeNode rightmost = curr.left;
                while (rightmost.right != null)
                    rightmost = rightmost.right;

                rightmost.right = curr.right;
                curr.right = curr.left;
                curr.left = null;
            }
            curr = curr.right;
        }
    }
}
```

```
        }
        curr = curr.right;
    }
}
```

Step 14: BST

PROBLEM:40

Problem Statement:

Introduction to Binary Search Tree

(Theoretical + Simple Insertion)

Theory:

A **Binary Search Tree (BST)** is a binary tree where for every node:

- Left child < Current node
 - Right child > Current node
-

Java Code:

```
java
CopyEdit
class TreeNode {
    int val;
    TreeNode left, right;
    TreeNode(int val) {
        this.val = val;
    }
}

class BST {
    public TreeNode insert(TreeNode root, int key) {
        if (root == null) return new TreeNode(key);
        if (key < root.val) root.left = insert(root.left, key);
        else root.right = insert(root.right, key);
        return root;
    }
}
```

PROBLEM:41

Problem Statement:

Search in a Binary Search Tree

Java Code:

```
java
CopyEdit
class Solution {
    public TreeNode searchBST(TreeNode root, int val) {
        if (root == null || root.val == val) return root;
        if (val < root.val) return searchBST(root.left, val);
        return searchBST(root.right, val);
    }
}
```

PROBLEM:42**Problem Statement:**

Find Minimum and Maximum in BST

Java Code:

```
java
CopyEdit
class Solution {
    public int findMin(TreeNode root) {
        while (root.left != null)
            root = root.left;
        return root.val;
    }

    public int findMax(TreeNode root) {
        while (root.right != null)
            root = root.right;
        return root.val;
    }
}
```

PROBLEM:43**Problem Statement:****Delete a Node in a BST**

Delete a node from the BST and maintain its properties.

Java Code:

```
java
CopyEdit
class Solution {
    public TreeNode deleteNode(TreeNode root, int key) {
        if (root == null) return null;

        if (key < root.val) root.left = deleteNode(root.left, key);
```



```

        else if (key > root.val) root.right = deleteNode(root.right, key);
    else {
        if (root.left == null) return root.right;
        if (root.right == null) return root.left;

        TreeNode minNode = getMin(root.right);
        root.val = minNode.val;
        root.right = deleteNode(root.right, minNode.val);
    }
    return root;
}

private TreeNode getMin(TreeNode node) {
    while (node.left != null) node = node.left;
    return node;
}
}

```

PROBLEM:44

Problem Statement:

Lowest Common Ancestor in BST

Find LCA of two nodes in a BST.

Java Code:

```

java
CopyEdit
class Solution {
    public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p,
    TreeNode q) {
        if (root.val > p.val && root.val > q.val)
            return lowestCommonAncestor(root.left, p, q);
        if (root.val < p.val && root.val < q.val)
            return lowestCommonAncestor(root.right, p, q);
        return root;
    }
}

```

PROBLEM:45

Problem Statement:

Kth Smallest Element in BST

Java Code:

```

java
CopyEdit
class Solution {
    int count = 0, result = -1;

```

```

    public int kthSmallest(TreeNode root, int k) {
        inorder(root, k);
        return result;
    }

    private void inorder(TreeNode node, int k) {
        if (node == null) return;
        inorder(node.left, k);
        count++;
        if (count == k) {
            result = node.val;
            return;
        }
        inorder(node.right, k);
    }
}

```

PROBLEM:46

Problem Statement:

Validate BST

Check if a binary tree is a valid Binary Search Tree.

Java Code:

```

java
CopyEdit
class Solution {
    public boolean isValidBST(TreeNode root) {
        return validate(root, Long.MIN_VALUE, Long.MAX_VALUE);
    }

    private boolean validate(TreeNode node, long min, long max) {
        if (node == null) return true;
        if (node.val <= min || node.val >= max) return false;
        return validate(node.left, min, node.val) && validate(node.right,
node.val, max);
    }
}

```

PROBLEM:47

Problem Statement:

Ceil in a Binary Search Tree

Find the smallest value in BST that is greater than or equal to the target.

Java Code:

```

java
CopyEdit
class Solution {

```

```
public int findCeil(TreeNode root, int key) {
    int ceil = -1;
    while (root != null) {
        if (root.val == key) return key;
        if (root.val < key) {
            root = root.right;
        } else {
            ceil = root.val;
            root = root.left;
        }
    }
    return ceil;
}
```

PROBLEM:48

Problem Statement:

Floor in a Binary Search Tree

Find the largest value in BST that is less than or equal to the target.

Java Code:

```
java
CopyEdit
class Solution {
    public int findFloor(TreeNode root, int key) {
        int floor = -1;
        while (root != null) {
            if (root.val == key) return key;
            if (root.val > key) {
                root = root.left;
            } else {
                floor = root.val;
                root = root.right;
            }
        }
        return floor;
    }
}
```

PROBLEM:49

Problem Statement:

Insert a Node in BST

Java Code:

```
java
CopyEdit
```

```

class Solution {
    public TreeNode insert(TreeNode root, int key) {
        if (root == null) return new TreeNode(key);
        if (key < root.val) root.left = insert(root.left, key);
        else root.right = insert(root.right, key);
        return root;
    }
}

```

PROBLEM:50

Problem Statement:

Delete a Node in Binary Search Tree

(Already given earlier as Problem 43 — consider renumbering if needed.)

PROBLEM:51

Problem Statement:

Find Kth Smallest/Largest Element in BST

Java Code (Kth Smallest):

```

java
CopyEdit
class Solution {
    int count = 0, result = -1;
    public int kthSmallest(TreeNode root, int k) {
        inorder(root, k);
        return result;
    }
    private void inorder(TreeNode node, int k) {
        if (node == null) return;
        inorder(node.left, k);
        count++;
        if (count == k) {
            result = node.val;
            return;
        }
        inorder(node.right, k);
    }
}

```

Java Code (Kth Largest):

```

java
CopyEdit
class Solution {
    int count = 0, result = -1;
    public int kthLargest(TreeNode root, int k) {
        reverseInorder(root, k);
        return result;
    }
}

```

```

    }
    private void reverseInorder(TreeNode node, int k) {
        if (node == null) return;
        reverseInorder(node.right, k);
        count++;
        if (count == k) {
            result = node.val;
            return;
        }
        reverseInorder(node.left, k);
    }
}

```

PROBLEM:52

Problem Statement:

Check if a tree is BST or BT

Java Code:

```

java
CopyEdit
class Solution {
    public boolean isBST(TreeNode root) {
        return check(root, Long.MIN_VALUE, Long.MAX_VALUE);
    }
    private boolean check(TreeNode node, long min, long max) {
        if (node == null) return true;
        if (node.val <= min || node.val >= max) return false;
        return check(node.left, min, node.val) && check(node.right,
node.val, max);
    }
}

```

PROBLEM:53

Problem Statement:

LCA in Binary Search Tree

(Already given as Problem 44.)

PROBLEM:54

Problem Statement:

Construct BST from Preorder Traversal

Java Code:

```

java
CopyEdit
class Solution {
    int index = 0;
    public TreeNode bstFromPreorder(int[] preorder) {
        return build(preorder, Integer.MAX_VALUE);
    }

    private TreeNode build(int[] preorder, int bound) {
        if (index == preorder.length || preorder[index] > bound) return
null;
        TreeNode root = new TreeNode(preorder[index++]);
        root.left = build(preorder, root.val);
        root.right = build(preorder, bound);
        return root;
    }
}

```

PROBLEM:55

Problem Statement:

Inorder Successor/Predecessor in BST

Java Code (Inorder Successor):

```

java
CopyEdit
class Solution {
    public TreeNode inorderSuccessor(TreeNode root, TreeNode p) {
        TreeNode succ = null;
        while (root != null) {
            if (p.val < root.val) {
                succ = root;
                root = root.left;
            } else {
                root = root.right;
            }
        }
        return succ;
    }
}

```

PROBLEM:56

Problem Statement:

Merge 2 BSTs

Java Code (Inorder + Merge + Build):

```

java

```

CopyEdit

```
class Solution {
    public List<Integer> merge(TreeNode root1, TreeNode root2) {
        List<Integer> list1 = new ArrayList<>();
        List<Integer> list2 = new ArrayList<>();
        inorder(root1, list1);
        inorder(root2, list2);
        return mergeSorted(list1, list2);
    }

    private void inorder(TreeNode root, List<Integer> list) {
        if (root == null) return;
        inorder(root.left, list);
        list.add(root.val);
        inorder(root.right, list);
    }

    private List<Integer> mergeSorted(List<Integer> a, List<Integer> b) {
        List<Integer> res = new ArrayList<>();
        int i = 0, j = 0;
        while (i < a.size() && j < b.size()) {
            if (a.get(i) < b.get(j)) res.add(a.get(i++));
            else res.add(b.get(j++));
        }
        while (i < a.size()) res.add(a.get(i++));
        while (j < b.size()) res.add(b.get(j++));
        return res;
    }
}
```

PROBLEM:57

Problem Statement:

Two Sum in BST | Pair with Sum K

Java Code:

java

CopyEdit

```
class Solution {
    public boolean findTarget(TreeNode root, int k) {
        Set<Integer> set = new HashSet<>();
        return dfs(root, set, k);
    }

    private boolean dfs(TreeNode node, Set<Integer> set, int k) {
        if (node == null) return false;
        if (set.contains(k - node.val)) return true;
        set.add(node.val);
        return dfs(node.left, set, k) || dfs(node.right, set, k);
    }
}
```

PROBLEM:58

Problem Statement:
Recover BST | Correct two nodes swapped

Java Code:

```
java
CopyEdit
class Solution {
    TreeNode first = null, second = null, prev = new
    TreeNode(Integer.MIN_VALUE);

    public void recoverTree(TreeNode root) {
        inorder(root);
        int temp = first.val;
        first.val = second.val;
        second.val = temp;
    }

    private void inorder(TreeNode root) {
        if (root == null) return;
        inorder(root.left);
        if (first == null && prev.val > root.val) first = prev;
        if (first != null && prev.val > root.val) second = root;
        prev = root;
        inorder(root.right);
    }
}
```

PROBLEM:59

Problem Statement:
Largest BST in Binary Tree

Java Code:

```
java
CopyEdit
class Solution {
    class Info {
        boolean isBST;
        int size, min, max;
        Info(boolean isBST, int size, int min, int max) {
            this.isBST = isBST;
            this.size = size;
            this.min = min;
            this.max = max;
        }
    }

    int maxSize = 0;

    public int largestBSTSubtree(TreeNode root) {
```



```

        postOrder(root);
        return maxSize;
    }

    private Info postOrder(TreeNode root) {
        if (root == null) return new Info(true, 0, Integer.MAX_VALUE,
Integer.MIN_VALUE);

        Info left = postOrder(root.left);
        Info right = postOrder(root.right);

        if (left.isBST && right.isBST && root.val > left.max && root.val <
right.min) {
            int size = left.size + right.size + 1;
            maxSize = Math.max(maxSize, size);
            return new Info(true, size, Math.min(root.val, left.min),
Math.max(root.val, right.max));
        }
        return new Info(false, 0, 0, 0);
    }
}

```

Congratulations on completing the PDF created by SYNTAX ERROR, also known as Abhishek Rathor! Your dedication and perseverance are truly commendable. For more insights and updates, feel free to connect on Instagram at SYNTAX ERROR. Keep up the great w